Maximilien Godin

Q-Learning: Ejercicio01

Descripción del código

El programa está dividido en dos archivos: environement.py define el entorno de Cliff Walking y train.py implementa Q-learning y lo entrena.

environement.py

Define la cuadrícula de 4×12 con la salida START = (3, 0) y la meta FINISH = (3, 11). El diccionario ACTION asocia cada acción (N, E, S, W) con su desplazamiento en fila y columna. Las funciones is_cliff e is_out indican si una casilla pertenece al acantilado (fila 3, columnas 1 a 10) o si está fuera del mapa.

La función **step(case, action)** calcula la nueva casilla y devuelve (nueva_casilla, recompensa, terminado):

si cae en el acantilado, regresa a START con recompensa −100 y el episodio continúa;

si la acción saca al agente del mapa, se queda en la misma casilla con −1;

si llega a la meta, recompensa −1 y el episodio termina;

en cualquier otro caso, recompensa −1.

El bloque **__main__** contiene los tres casos de prueba del enunciado y cuatro más, con assert.

train.py

Fija los parámetros del ejercicio (α = 0.5, γ = 1, ε = 0.1, 500 episodios, tope de 200 pasos y semilla 42). MOVES es la lista ordenada de acciones (N, E, S, W), y ese orden define el desempate.

empty_q() crea la tabla Q: un valor por casilla no terminal y por acción, inicializado en 0. La meta y el acantilado no aparecen en la tabla.

best_move() devuelve la acción de mayor valor Q; en caso de empate conserva la primera de la lista.

epsilon_greedy() elige una acción aleatoria con probabilidad ε y, si no, la mejor acción.

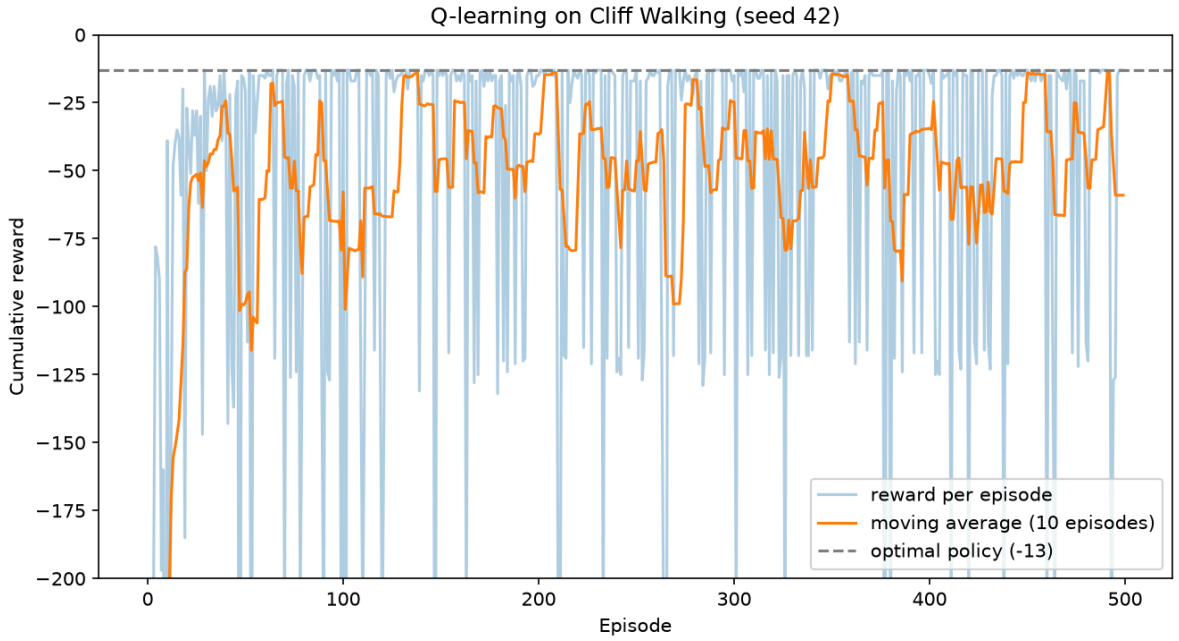
train() recorre los 500 episodios desde START. En cada paso elige una acción, llama a step y actualiza Q(s, a) con el blanco r + γ · max Q(s', ·), que vale solo r si s' es la meta. Devuelve la tabla Q y la recompensa acumulada de cada episodio.

evaluate() recorre la política greedy (ε = 0) desde START y devuelve el camino, el número de pasos y el retorno.

print_policy() imprime la política como flechas sobre la cuadrícula, y **moving_average()** calcula la media móvil de 10 episodios.

El bloque **__main__** entrena, muestra la política greedy (camino de 13 pasos, retorno −13) y guarda la gráfica de recompensa por episodio en q_learning_curve.png.

Gráfico de la recompensa por episodio



Reporte: Q-learning en Cliff Walking

Implementé Q-learning tabular con política ε-greedy en la cuadrícula de 4×12 del libro de Sutton y Barto. Usé α = 0.5, γ = 1, ε = 0.1, 500 episodios, Q inicial en 0 y semilla 42. En los empates elegí la primera acción en el orden N, E, S, W. Con la Q entrenada, la política greedy (ε = 0) recorre el camino de 13 pasos y obtiene un retorno de −13.

1. ¿Por qué el camino de 13 pasos, pegado al acantilado, es óptimo?

Porque cada paso cuesta −1 y no hay descuento (γ = 1), así que el retorno de un camino es simplemente menos su número de pasos. El mejor retorno es el del camino más corto de S a G. Desde (3, 0) hay que subir una fila, avanzar once columnas hacia el este y bajar a G, lo que da 13 pasos y un retorno de −13. Ese camino va por la fila 2, justo encima del acantilado. Ninguno más corto es posible, y cualquier ruta más alejada del borde añade pasos y por tanto empeora el retorno. Como la política greedy no explora, nunca da un paso al azar hacia el acantilado, así que el camino corto no tiene riesgo.

Maximilien Godin

2. ¿Por qué la curva de entrenamiento queda por debajo de −13 aunque la política greedy ya sea óptima?

Porque durante el entrenamiento el agente sigue la política ε-greedy, no la greedy. Con ε = 0.1, una de cada diez acciones es aleatoria. Al caminar por la fila 2, un solo paso aleatorio hacia el sur lo hace caer al acantilado (−100) y lo manda de vuelta a S, y el episodio tiene que empezar de nuevo desde ahí. Por eso en la gráfica la media móvil se queda en un rango aproximado de −25 a −60 en vez de pegarse a −13, y hay episodios sueltos con recompensas por debajo de −100. Aun así, muchos episodios llegan justo a −13, lo que muestra que la política aprendida ya es buena. El −13 solo se mide en la evaluación con ε = 0.

3. En el paso que cae al acantilado, ¿cuál es s' en el backup de Q-learning y por qué no se trata como estado terminal?

Cuando el agente cae, el entorno lo devuelve a la casilla de inicio, así que s' = S = (3, 0), y la recompensa es −100. El acantilado no es un estado terminal porque el episodio no termina: el agente sigue jugando desde S. Por eso S es un estado normal y el blanco del update es $r + \gamma \cdot \max Q(S, \cdot) = -100 + \max Q(S, \cdot)$, en lugar de $r + 0$, que sería lo que se usa al llegar a G. Así, el valor de Q(s, a) para esa acción refleja el castigo de −100 más lo que todavía le queda por recorrer para llegar a G desde S.